AAPL Financial Analytics Pipeline

API Integration, Rule-Based Market Classification, and SQLite Reporting

Apple Inc. (AAPL)

Data snapshot: April 17, 2026 - May 14, 2026

Carlos Herrera

Financial Analytics Portfolio Project

Executive Summary

This project builds an end-to-end financial analytics pipeline for Apple Inc. (AAPL). The system accepts either a saved Alpha Vantage daily time-series response or a live API request, validates the nested JSON structure, converts price and volume fields to numeric types, calculates daily movement and volatility metrics, applies transparent business rules, and stores the processed records in SQLite for repeatable SQL analysis.

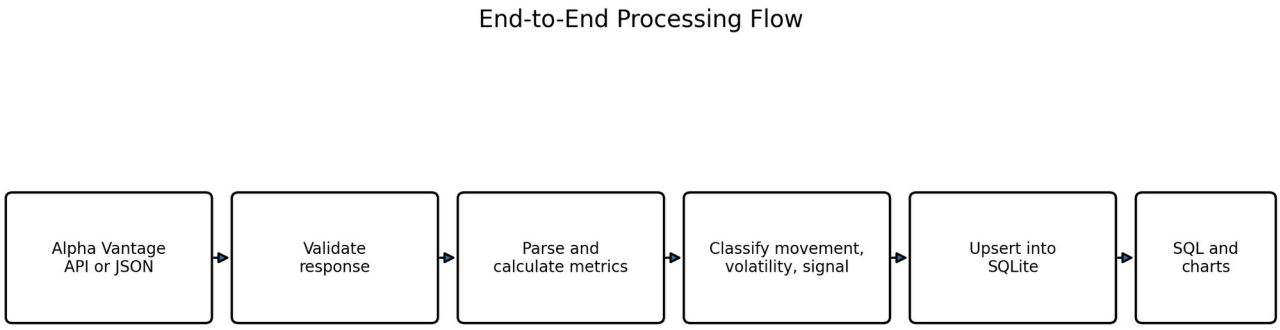
The included reproducible sample covers 20 trading days from April 17, 2026 through May 14, 2026. The rule-based process produced 10 Buy classifications, 7 Sell classifications, and 3 Hold classifications. Twelve days were labeled Moderate volatility, seven Low volatility, and one High volatility. The average closing price was $280.05, and May 1, 2026 had the highest intraday range at 3.17% of the opening price.

The labels are descriptive, not predictive. They summarize same-day open-to-close behavior and apply a conservative Hold override when intraday volatility is High. The project demonstrates API integration, JSON parsing, algorithmic business rules, reusable Python functions, data-quality validation, relational database design, SQL analysis, and reproducible reporting.

Key Outcomes

Metric	Result
Records analyzed	20
Buy / Sell / Hold	10 / 7 / 3
Low / Moderate / High volatility	7 / 12 / 1
Average closing price	$280.05
Highest-volatility day	May 1, 2026 (3.17%)
Primary data store	SQLite with composite key

System Architecture



The offline JSON option makes the project reproducible without exposing an API key. For live execution, the key is read from the ALPHA_VANTAGE_API_KEY environment variable rather than being hard-coded in source control.

1. Business Problem and User Scenario

The intended user is a portfolio or market analyst who needs a lightweight way to summarize daily AAPL price behavior. The system converts raw daily price data into consistent metrics and labels so an analyst can quickly identify positive or negative sessions, compare volatility conditions, and query the resulting history in SQL.

- Automate ingestion of open, high, low, close, and volume fields.
- Standardize calculations across trading days.
- Apply transparent and auditable decision rules.
- Prevent duplicate symbol-date records in the database.
- Produce reusable SQL summaries and portfolio-ready visuals.

2. Data Source and Fields

The project uses the Alpha Vantage TIME_SERIES_DAILY response format. A saved JSON response is included for reproducibility. The daily time-series object uses the trading date as the key and stores five raw market fields as strings.

Source field	Analytical meaning	Stored type
1. open	Opening price	REAL
2. high	Highest intraday price	REAL
3. low	Lowest intraday price	REAL
4. close	Closing price	REAL
5. volume	Shares traded	INTEGER

3. Calculated Metrics

Metric	Formula	Purpose
Price change	Close - Open	Absolute open-to-close movement
Percent change	(Close - Open) / Open x 100	Normalizes movement across price levels
Daily range	High - Low	Absolute intraday spread
Daily range percent	(High - Low) / Open x 100	Normalizes intraday volatility

4. Business Rules and Classification Logic

Rule category	Condition	Output
Movement	Percent change > 0.5%	Positive
Movement	Percent change < -0.5%	Negative
Movement	-0.5% to 0.5%	Neutral
Volatility	Daily range percent < 1.5%	Low
Volatility	1.5% through 3.0%	Moderate
Volatility	Daily range percent > 3.0%	High
Signal	Volatility is High	Hold override
Signal	Positive and volatility is not High	Buy
Signal	Negative and volatility is not High	Sell
Signal	Neutral	Hold

The High-volatility Hold override is a conservative risk-control layer. It prevents a Buy classification on a session with unusually wide intraday movement. The resulting signal remains a summary of observed behavior rather than a forward-looking trading recommendation.

5. Python Implementation

The cleaned implementation separates data acquisition, validation, transformation, classification, database persistence, SQL analysis, chart creation, and command-line execution. Error handling covers failed HTTP requests, invalid JSON, API rate-limit messages, missing time-series sections, missing fields, and non-numeric values.

Function	Responsibility
get_stock_data()	Call Alpha Vantage using a symbol and environment-based API key.
validate_response()	Detect API messages and confirm the daily time-series object exists.
calculate_metrics()	Convert raw fields and calculate price and range metrics.
parse_daily_records()	Process the most recent valid records up to the requested limit.
classify_movement()	Assign Positive, Negative, or Neutral.
classify_volatility()	Assign Low, Moderate, or High.
generate_signal()	Assign Buy, Sell, or Hold with the volatility override.
create_database()	Create the SQLite table and composite primary key.
insert_records()	Upsert records without duplicate symbol-date rows.
run_analytics_queries()	Return the core SQL summaries.

Reproducible Command

```
python src/aapl_financial_pipeline.py \
  --input-json data/sample_aapl_daily.json \
  --database database/aapl_stock_analysis.db \
  --limit 20
```

6. Database Design

The stock_analysis table stores one analytical row per symbol and trading date. A composite primary key on (symbol, trade_date) prevents duplicates, while SQLite upsert logic updates an existing record if the pipeline is rerun for the same date.

Column group	Fields
Identifiers	symbol, trade_date
Raw prices	open_price, high_price, low_price, close_price, volume
Calculated metrics	price_change, percent_change, daily_range, daily_range_percent
Classifications	movement_label, volatility_label, signal

7. Analytical SQL

The repository includes a separate SQL file so the database logic can be reviewed without opening the SQLite file. The five core analyses are:

- Count Buy, Sell, and Hold classifications.
- Calculate average percent change by signal type.
- Identify the highest-volatility trading day.
- List all High-volatility records.
- Calculate average closing price by symbol.

```
SELECT signal, COUNT(*) AS signal_count
FROM stock_analysis
GROUP BY signal
ORDER BY signal_count DESC, signal;

SELECT signal, ROUND(AVG(percent_change), 2) AS avg_percent_change
FROM stock_analysis
GROUP BY signal
ORDER BY signal;
```

SQL Results

Signal	Days	Average percent change
Buy	10	1.41%
Hold	3	0.34%
Sell	7	-0.86%

May 1, 2026 was the only High-volatility session in the sample. Its 3.17% daily range triggered the Hold override. This is consistent with the explicitly defined risk-control rule.

8. Results and Interpretation

The 20-day sample produced mixed but net-positive daily classifications. Ten sessions closed more than 0.5% above the open without High volatility, seven closed more than 0.5% below the open without High volatility, and three were held because movement was Neutral or volatility was High.

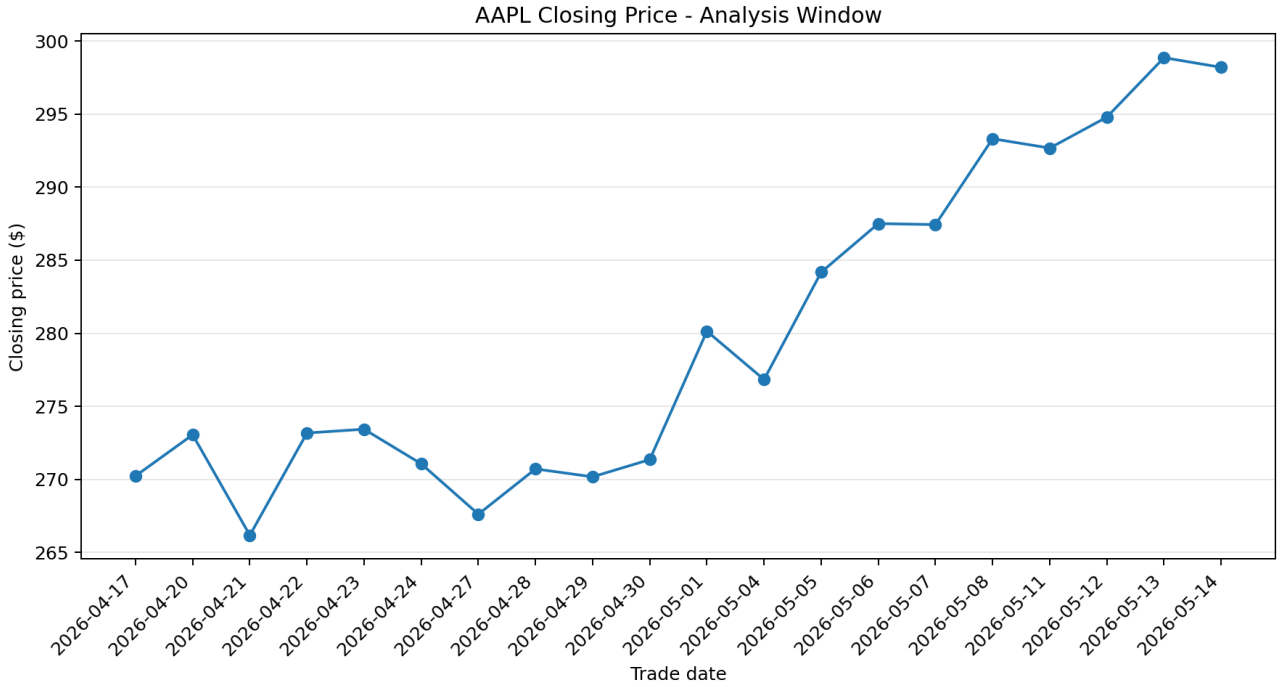


Figure 1. AAPL closing prices during the included analysis window.

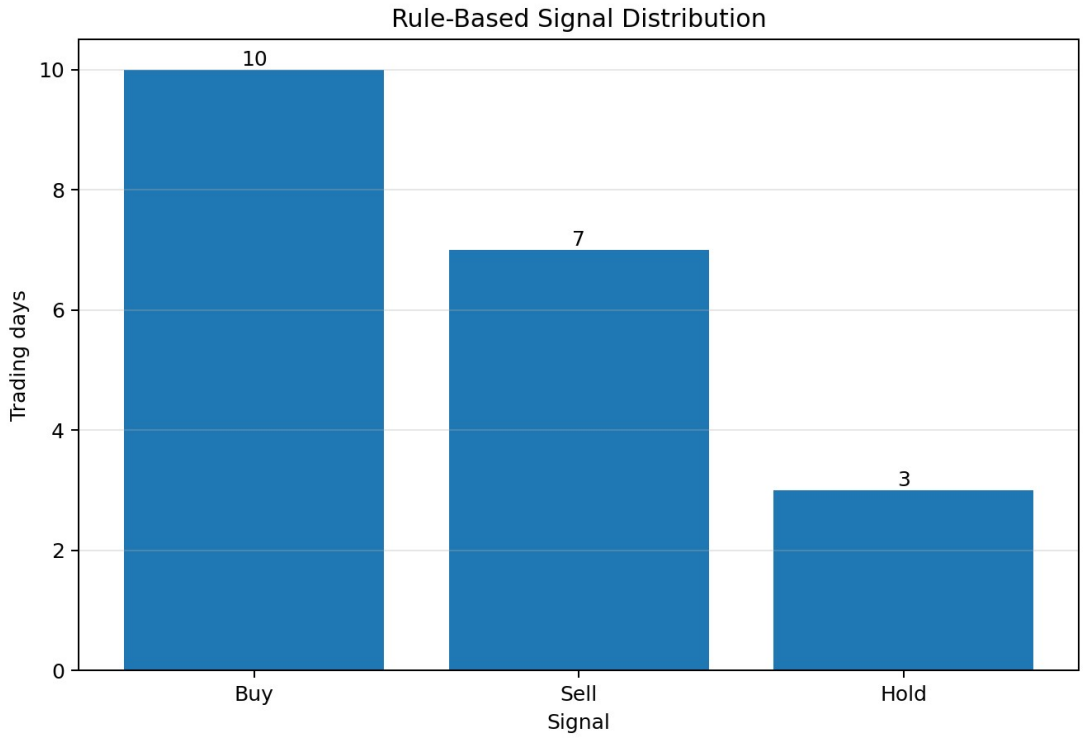


Figure 2. Distribution of rule-based daily classifications.

9. Limitations and Responsible Use

- The model uses same-day open, high, low, and close values; it does not predict future returns.
- Thresholds are interpretable rules selected for demonstration, not statistically optimized trading parameters.
- The sample is limited to 20 trading days and one company.
- The analysis excludes earnings, fundamentals, news, macroeconomic conditions, transaction costs, and portfolio constraints.
- Live execution depends on API availability and rate limits.
- Buy, Sell, and Hold labels should not be interpreted as investment advice.

10. Recommended Extensions

- Compare multiple securities using the same normalized calculations.
- Add rolling returns, moving averages, average true range, and volume-based features.
- Separate descriptive classifications from genuinely predictive targets and evaluate on out-of-sample data.
- Add automated tests for parsing, classification boundaries, database upserts, and API failure modes.
- Schedule pipeline runs and publish an interactive dashboard from the SQLite outputs.

Conclusion

The project demonstrates a complete analytics workflow rather than an isolated script: external data acquisition, JSON validation, numeric transformation, algorithmic business rules, reusable Python design, SQLite persistence, SQL analysis, and communication through charts and a written report. Its primary value is technical integration and interpretability. The system is deliberately positioned as a market-behavior classification tool, not a forecasting or trading product.

Repository Contents

Path	Purpose
src/aapl_financial_pipeline.py	Main API, processing, classification, and database pipeline.
src/generate_charts.py	Creates portfolio visuals from the SQLite database.
sql/schema.sql	Database table definition.
sql/analytics_queries.sql	Five reusable analytical queries.
data/sample_aapl_daily.json	Saved API response for offline reproducibility.
database/aapl_stock_analysis.db	Sample SQLite output with 20 processed records.